

scrawl



FERGOUS Abdelhak

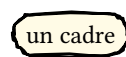
des formes à main levée, en Typst pur — aucun greffon, aucune dépendance

chaque figure de ce document est produite par le code affiché à sa gauche

Les trois raccourcis

Ils mesurent leur contenu : le cadre épouse ce qu'on y met.

```
#grid(columns: 3, column-gutter: 5mm, align: horizon,
  scrawl-box(fill: rgb("#fffbe6"))[un cadre],
  scrawl-ellipse(paint: rgb("#166534"))[cerclé],
  [du texte #scrawl-underline[souligné ici],
])
```



un cadre



cerclé

du texte souligné ici

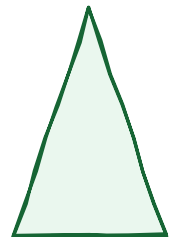
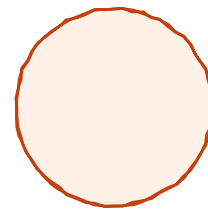
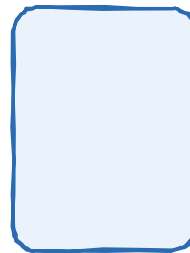
```
Et un #hl[surligneur],
en #hl(colour: rgb("#B7E3FF"))[deux teintes].
```

Et un surligneur, en deux teintes.

Le canevas

Coordonnées en centimètres, **y vers le haut**. Le corps reçoit six fonctions déjà liées au canevas : shape, lines, region, rough, label, arrow.

```
#scrawl(width: 8.4cm, height: 3.6cm, (shape, ..) => {
  shape(rounded-rect-pts((0.2, 0.2), (2.6, 3.4), radius: 0.3),
    paint: rgb("#2B6CB0"), fill: rgb("#EAF2FB"),
    weight: 1.2pt)
  shape(circle-pts((4.4, 1.8), 1.3),
    paint: rgb("#C2410C"), fill: rgb("#FFF1E7"))
  shape(((6.2, 0.3), (8.2, 0.3), (7.2, 3.3)),
    paint: rgb("#166534"), fill: rgb("#EAF7EE"),
    weight: 1.2pt)
})
```



Les contours sont des tableaux de points

Tout ce qui prend un contour prend un simple tableau de (x, y) : les constructeurs ne sont qu'une commodité.

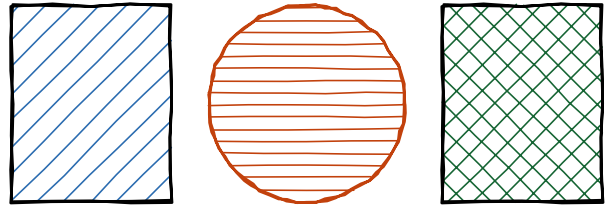
```
#scrawl(width: 8.2cm, height: 3.6cm, (shape, ..) => {
  // une étoile, calculée
  let star = range(10).map(i => {
    let a = 90deg - i * 36deg
    let r = if calc.rem(i, 2) == 0 { 1.5 } else { 0.62 }
    (2.0 + r * calc.cos(a), 1.8 + r * calc.sin(a))
  })
  shape(star, paint: rgb("#B45309"), fill: rgb("#FEF3C7"),
    weight: 1.2pt)
  // un demi-disque, par un arc
  shape(arc-pts((5.6, 1.2), 1.4, 0, 180) + ((4.2, 1.2),),
    paint: rgb("#7C3ABA"), fill: rgb("#F3E8FF"),
    weight: 1.2pt)
})
```



Les hachures

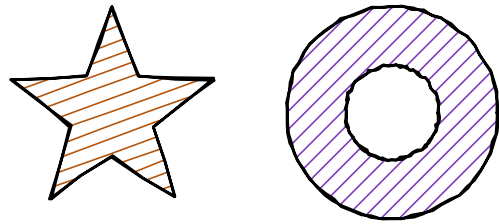
`hatching(...)` se donne à `fill:`, là où irait une couleur — parce que la hachure **est** le remplissage. Il n'y a donc pas moyen de demander un fond rouge hachuré avec un pas nul : il n'y a qu'une chose à dire.

```
#scrawl(width: 8.2cm, height: 3.2cm, (shape, ..) => {
  shape(rect-pts((0.2, 0.3), (2.3, 2.9)), paint: black,
    fill: hatching(rgb("#2B6CB0")))
  shape(circle-pts((4.1, 1.6), 1.3), paint: rgb("#C2410C"),
    fill: hatching(rgb("#C2410C"), angle: 0deg, gap: 0.16))
  shape(rect-pts((5.9, 0.3), (8.0, 2.9)), paint: black,
    fill: hatching(rgb("#166534"), cross: true, gap: 0.24))
})
```



Le balayage suit la règle **pair-impair** : un contour concave se hachure juste, et un second contour perce un trou.

```
#scrawl(width: 8.2cm, height: 3.2cm, (shape, lines, ..) => {
  let star = range(10).map(i => {
    let a = 90deg - i * 36deg
    let r = if calc.rem(i, 2) == 0 { 1.4 } else { 0.58 }
    (1.7 + r * calc.cos(a), 1.6 + r * calc.sin(a))
  })
  shape(star, paint: black,
    fill: hatching(rgb("#B45309"), angle: 20deg, gap: 0.17))
  // un anneau : le disque, puis le trou — une seule région
  let o = circle-pts((5.4, 1.6), 1.4)
  let i = circle-pts((5.4, 1.6), 0.62)
  place(top + left, lines(hatch-pts((o, i), gap: 0.17),
    stroke: (paint: rgb("#7C3ABA"), thickness: 0.6pt)))
  shape(o, paint: black)
  shape(i, paint: black)
})
```



`backdrop` : glisse un aplat sous les traits — sans lui, du texte posé sur des hachures est illisible.

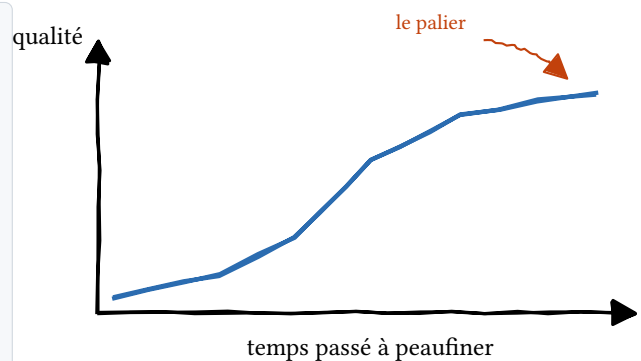
```
#grid(columns: 2, column-gutter: 4mm, align: horizon,
  scrawl-box(fill: hatching(rgb("#B45309"),
    gap: 0.15))[illisible],
  scrawl-box(fill: hatching(rgb("#B45309"), gap: 0.15,
    backdrop: white))[lisible],
)
```



Un graphique, façon tableau noir

`arrow` trace le trait et sa pointe ; `label` pose du texte aux coordonnées du canevas, sans conversion à faire soi-même. `bend` : cintre la flèche — la pointe suit alors la tangente, pas la corde.

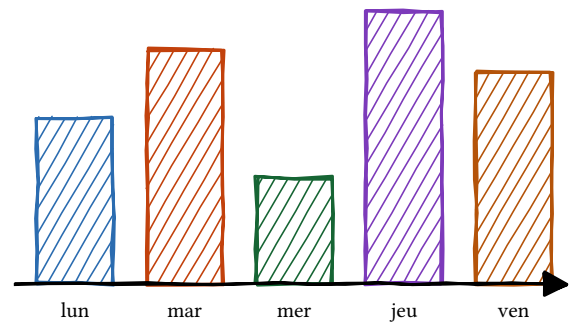
```
#scrawl(width: 8.2cm, height: 4.8cm, roughness: 1.1,
  (shape, lines, region, rough, label, arrow) => {
  arrow((0.8, 0.7), (7.9, 0.7), weight: 1.2pt)
  arrow((0.8, 0.7), (0.8, 4.3), weight: 1.2pt)
  shape(((1.0, 0.9), (2.4, 1.2), (3.4, 1.7), (4.4, 2.7),
    (5.6, 3.3), (6.6, 3.5), (7.4, 3.6))),
    paint: rgb("#2B6CB0"), weight: 1.6pt, closed: false)
  label((4.4, 0.2), [temps passé à peaufiner])
  label((0.6, 4.3), [qualité], anchor: right + horizon)
  label((5.2, 4.5),
    text(8pt, fill: rgb("#C2410C"))[le palier])
  arrow((5.9, 4.3), (7.0, 3.8), bend: 0.18,
    paint: rgb("#C2410C"), weight: 0.8pt)
})
```



Des barres

Rien de spécial : un rectangle par barre, et une étiquette dessous. Une graine différente par barre, sinon les cinq tremblent à l'identique.

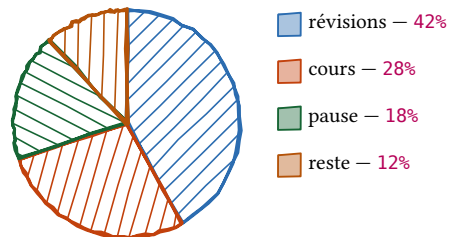
```
#scrawl(width: 8.2cm, height: 4.4cm,
  (shape, l, r, ro, label, arrow) => {
  let data = (("lun", 2.2), ("mar", 3.1), ("mer", 1.4),
    ("jeu", 3.6), ("ven", 2.8))
  let cols = (rgb("#2B6CB0"), rgb("#C2410C"), rgb("#166534"),
    rgb("#7C3ABA"), rgb("#B45309"))
  for (i, d) in data.enumerate() {
    let x = 1.0 + i * 1.45
    shape(rect-pts((x, 0.8), (x + 1.0, 0.8 + d.at(1))),
      paint: cols.at(i), fill: hatching(cols.at(i), gap: 0.15,
        angle: 60deg), weight: 1.1pt, seed: 7 + i * 5)
    label((x + 0.5, 0.4), text(8pt, d.at(0)))
  }
  arrow((0.7, 0.8), (8.0, 0.8), weight: 1.1pt)
})
```



Un camembert

Une part est un arc refermé sur son centre. La hachure tourne avec la part, ce qui les distingue même imprimées en noir et blanc.

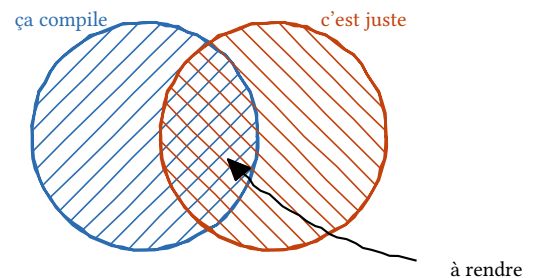
```
#scrawl(width: 8.2cm, height: 3.6cm,
  (shape, l, r, ro, label, ..) => {
  let parts = ((42%, "révisions", rgb("#2B6CB0")),
    (28%, "cours", rgb("#C2410C")),
    (18%, "pause", rgb("#166534")),
    (12%, "reste", rgb("#B45309")))
  let a = 90.0
  for (i, (f, nom, col)) in parts.enumerate() {
    let b = a - f / 100% * 360
    shape(arc-pts((2.0, 1.8), 1.5, a, b, n: 26)
      + ((2.0, 1.8)),
      paint: col, weight: 1.1pt, seed: 3 + i * 9,
      fill: hatching(col, gap: 0.17, angle: (a - 45) * 1deg))
    label((4.4, 3.1 - i * 0.62), text(8pt)[#nom - #f],
      anchor: left + horizon)
    shape(rect-pts((4.0, 2.95 - i * 0.62),
      (4.3, 3.25 - i * 0.62)),
      paint: col, fill: col.lighten(60%), weight: 0.7pt)
    a = b
  }
})
```



Un diagramme de Venn

Deux disques translucides, l'intersection annotée par une flèche cintrée.

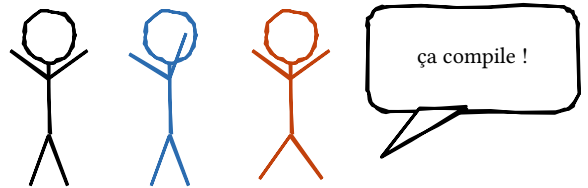
```
#scrawl(width: 8.2cm, height: 4.2cm,
  (shape, l, r, ro, label, arrow) => {
  shape(circle-pts((3.0, 2.4), 1.5), paint: rgb("#2B6CB0"),
    fill: hatching(rgb("#2B6CB0"), angle: 45deg, gap: 0.2))
  shape(circle-pts((4.7, 2.4), 1.5), paint: rgb("#C2410C"),
    fill: hatching(rgb("#C2410C"), angle: -45deg, gap: 0.2))
  label((1.9, 3.9),
    text(8pt, fill: rgb("#2B6CB0"))[ça compile])
  label((5.9, 3.9),
    text(8pt, fill: rgb("#C2410C"))[c'est juste])
  arrow((6.6, 0.75), (4.1, 2.1), bend: 0.16, weight: 0.8pt)
  label((8.0, 0.6), text(8pt)[à rendre],
    anchor: right + horizon)
})
```



Des bonshommes allumettes

Rien dans le paquet ne connaît les bonshommes : c'est un cercle et quatre segments. Les angles des membres sont les paramètres d'une fonction locale.

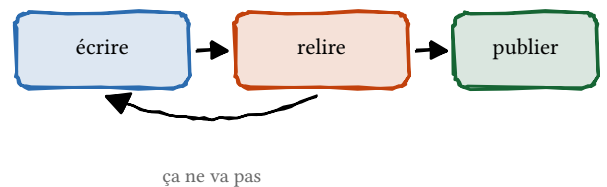
```
#scrawl(width: 8.2cm, height: 4.4cm,
  (shape, l, r, ro, label, arrow) => {
  let gus(x, y, col: black, bras: (35deg, 145deg),
    jambes: (250deg, 290deg)) = {
    let trait(a, b) = shape((a, b), paint: col,
      closed: false, weight: 1.1pt)
    shape(circle-pts((x, y), 0.34, n: 26), paint: col,
      weight: 1.1pt)
    trait((x, y - 0.34), (x, y - 1.3))
    for a in bras {
      trait((x, y - 0.55), (x + 0.62 * calc.cos(a),
        y - 0.55 + 0.62 * calc.sin(a)))
    }
    for a in jambes {
      trait((x, y - 1.3), (x + 0.72 * calc.cos(a),
        y - 1.3 + 0.72 * calc.sin(a)))
    }
  }
  gus(1.0, 3.9)
  gus(2.6, 3.9, col: rgb("#2B6CB0"), bras: (70deg, 150deg))
  gus(4.2, 3.9, col: rgb("#C2410C"), jambes: (235deg, 305deg))
  shape(rounded-rect-pts((5.2, 2.9), (8.0, 4.3), radius: 0.3),
    paint: black, fill: white, weight: 1.1pt)
  shape(((5.9, 2.95), (5.4, 2.3), (6.5, 2.95)),
    paint: black, fill: white, weight: 1.1pt)
  label((6.6, 3.6), text(8.5pt)[ça compile !])
})
```



Un schéma relié

Des boîtes, des flèches entre elles : le diagramme de tableau blanc. La flèche de retour est cintrée pour ne pas passer sous les boîtes.

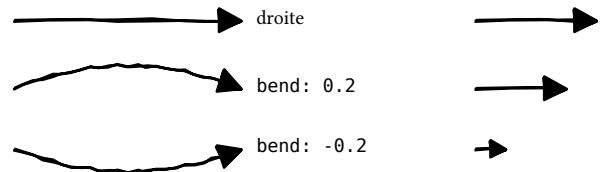
```
#scrawl(width: 8.2cm, height: 3.6cm,
  (shape, l, r, ro, label, arrow) => {
  let boite(x, y, w, h, txt, col) = {
    shape(rounded-rect-pts((x, y), (x + w, y + h),
      radius: 0.2),
    paint: col, fill: col.lighten(84%), weight: 1.1pt)
    label((x + w / 2, y + h / 2), text(8.5pt, txt))
  }
  boite(0.3, 2.1, 2.3, 1.0, [écrire], rgb("#2B6CB0"))
  boite(3.2, 2.1, 2.3, 1.0, [relire], rgb("#C2410C"))
  boite(6.1, 2.1, 1.9, 1.0, [publier], rgb("#166534"))
  arrow((2.7, 2.6), (3.1, 2.6), weight: 1pt)
  arrow((5.6, 2.6), (6.0, 2.6), weight: 1pt)
  arrow((4.3, 2.0), (1.5, 2.0), bend: 0.22, weight: 0.9pt)
  label((2.9, 0.9),
    text(7.5pt, fill: rgb("#666"))[ça ne va pas])
})
```



Les flèches

La pointe est plafonnée à 55 % de la longueur : sans cela, entre deux boîtes espacées de 4 mm, il ne reste qu'un triangle et deux pixels de trait.

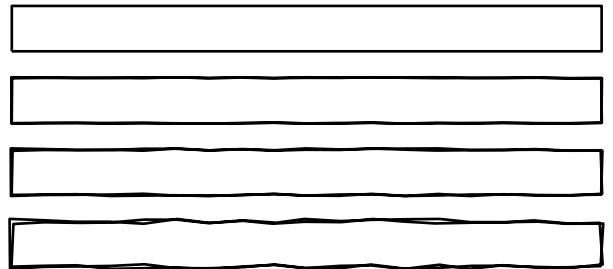
```
#scrawl(width: 8.2cm, height: 2.6cm,
  (shape, l, r, ro, label, arrow) => {
  arrow((0.3, 2.1), (3.3, 2.1))
  arrow((0.3, 1.2), (3.3, 1.2), bend: 0.2)
  arrow((0.3, 0.4), (3.3, 0.4), bend: -0.2)
  label((3.5, 2.1), text(7.5pt)[droite],
    anchor: left + horizon)
  label((3.5, 1.2), text(7.5pt)[`bend: 0.2`],
    anchor: left + horizon)
  label((3.5, 0.4), text(7.5pt)[`bend: -0.2`],
    anchor: left + horizon)
  // une flèche courte : la pointe se réduit avec elle
  arrow((6.4, 0.4), (6.8, 0.4), weight: 1pt)
  arrow((6.4, 1.2), (7.6, 1.2), weight: 1pt)
  arrow((6.4, 2.1), (8.0, 2.1), weight: 1pt)
})
```



Le degré de tremblé

roughness va de zéro — une règle — à un gribouillage. hand: false supprime le tremblé sans changer la géométrie.

```
#stack(dir: ttb, spacing: 1mm,
  ..(0, 0.5, 1.2, 2.5).map(r => scrawl(
    width: 8cm, height: 0.85cm, roughness: r, hand: r > 0,
    (shape, ..) => shape(rect-pts((0.1, 0.12), (7.9, 0.72)),
      paint: black),
  )))
```



L'amortissement

Une longue règle tremble proportionnellement moins qu'un petit cadre : juste pour un formulaire, faux pour un croquis. damping: false l'annule.

```
#grid(columns: 2, column-gutter: 3mm,
  scrawl(width: 4cm, height: 1.1cm, roughness: 2.0,
    (shape, ..) => {
      shape(rect-pts((0.1, 0.15), (3.9, 0.95)), paint: black)
    }
  ),
  scrawl(width: 4cm, height: 1.1cm, roughness: 2.0,
    (shape, ..) => {
      shape(rect-pts((0.1, 0.15), (3.9, 0.95)), paint: black,
        damping: false)
    }
  ),
)
```



Le déterminisme

Même seed, même tremblé, à chaque compilation : un document se reconstruit à l'identique. Les deux premiers cadres partagent leur graine.

```
#grid(columns: 4, column-gutter: 2.5mm,
  ..(1, 1, 2, 3).map(s => scrawl(
    width: 1.9cm, height: 1.2cm, seed: s, roughness: 1.4,
    (shape, ..) => shape(
      rounded-rect-pts((0.1, 0.1), (1.8, 1.1), radius: 0.15),
      paint: black),
  )))
```



Le tremblé, à nu

rough-amp donne l'amplitude d'un contour, jitter déplace des points : ce sont les deux moitiés du moteur. Les hachures s'en servent pour trembler **comme** la forme qui les contient — sinon un court segment près d'un coin s'agiterait plus que le long qui traverse, et le remplissage aurait l'air trié par longueur.

```
#scrawl(width: 8.2cm, height: 2.6cm, (shape, lines, ..) => {  
  let base = ((0.3, 1.3), (7.9, 1.3))  
  let pts = resample(base, step: 0.3, closed: false)  
  for (i, a) in (0.05, 0.15, 0.4).enumerate() {  
    place(top + left, lines(  
      jitter((pts.map(p =>  
        (p.at(0), p.at(1) + 0.75 - i * 0.7)),),  
        seed: 5 + i, amp: a),  
      flip: 2.6cm, stroke: 0.9pt + rgb("#2B6CB0"))  
    )  
  }  
})
```



Chaque figure de ce document est produite par le code affiché à sa gauche : demo évalue la même source qu'il imprime.